Lab 13: Authentication

Protect your application with a login form, a cookie and a session

Learning Objectives

By the end of this lab you will be able to:

- Explain what authentication, a cookie and a session are.
- Create a login form for your application.
- Create a session ID when a user logs in and store it in a cookie.
- Protect the application so only logged-in users can see the students' information.

Key Concepts

> **Authentication:** Authentication is the process of checking who a user is before giving them access to a system. The user proves their identity, usually with a username and password. If the details are correct, the user is allowed in. Authentication is not the same as authorization: authentication checks who you are, authorization decides what you are allowed to do.
>
> **Cookie:** A cookie is a small piece of data that a website stores in the user's browser. The server creates the cookie and sends it in a `Set-Cookie` response header. The browser saves it and sends it back to the server in the `Cookie` request header with every later request to the same site. Cookies are used to remember things like a session ID or user preferences. A cookie with no expiry date is a session cookie, and the browser deletes it when the browser is closed. This lab uses a session cookie.
>
> **Session:** A session is a record kept on the server that shows a user is logged in. Each session has a unique session ID. The server sends the ID to the browser in a cookie, and checks it on every request. In this lab the session IDs are stored in memory, so they are lost when the server restarts.

How the login works

1. The user opens a page. The browser has no session cookie yet.
2. The server checks the request with `isLoggedIn()`. There is no valid session, so the server redirects the user to `/login`.
3. The user fills in the login form and submits it.
4. The login controller checks the username and password. If they are correct, it creates a session ID, sends it to the browser in a cookie and redirects to the home page.
5. The browser sends the cookie with every later request, so the server knows the user is logged in.

Lab Steps

You will create or change the following files:

File	What it does
`views/login.js`	Creates the login form (HTML).
`assets/style.css`	Styles the login form.
`tasks/session.js`	Creates sessions and checks if a user is logged in.
`controllers/login.js`	Handles the login form and sets the cookie.
`server.js`	Adds the /login route and protects the rest of the app.

Step 1: Create the login view

In the **views** folder, create a file `login.js` and paste the following code.

views/login.js

```
export default function loginView(){

  return `
    <form action="/login" method="POST" id="loginform">
      <label for="usrname">Username</label>
      <input type="text" id="usrname" name="username">
      <label for="pass">Password</label>
      <input type="password" id="pass" name="password">
      <button>Sign In</button>
    </form>
  `;

}
```

Step 2: Add the login form styles

Add these additional styles to the `style.css` file in the **assets** folder.

assets/style.css

```
#loginform{
  display: grid;
  grid-template-columns: auto 1fr;
  gap: 0.5rem;
  place-self: center;
  font-family: system-ui;

  background: linear-gradient(#222, #555);
  color: white;
  padding: 2rem;

  button{
    grid-column: 2;
    place-self:inherit
  }
}
```

Step 3: Create the session file

In the **tasks** folder, create a file `session.js` and paste the following code.

tasks/session.js

```
const sessions = new Set();

export default function isLoggedIn(request){

  const cookie = request.headers.get("cookie") || "";
  const sessionId = cookie.split("session_id=")[1];

  console.log(sessionId)
  return sessions.has(sessionId)

}

export function login(){
  const sessionId = crypto.randomUUID();
  sessions.add(sessionId)
  return sessionId;
}
```

- `login()` creates a random session ID and saves it.
- `isLoggedIn()` reads the session ID from the cookie and checks if it was saved.

Step 4: Create the login controller

In the **controllers** folder, create a file `login.js` and paste the following code.

controllers/login.js

```
import render from "../render.js";
import { logIn } from "../tasks/session.js";
import loginView from "../views/login.js";

export default async function loginController({request}){

    if(request.method==="POST"){
        const form = await request.formData();
        const username = form.get("username");
        const password = form.get("password");
        if(username=="admin"&& password=="1234")
        {
            const sessionId = logIn();
            const headers = new Headers();
            headers.set("Location", "/")
            headers.set("Set-Cookie", `session_id=${sessionId}; Path=/`)
            return new Response(null, {status:302, headers})
        }

        return new Response("Username or Password is Incorrect")
    }

    return render(loginView);
}
```

- If the form is submitted (POST) with the correct username and password, the server sets the session cookie and redirects to the home page.

- If the details are wrong, the server sends a 401 response with an error message.
- If the page is opened normally (GET), the server shows the login form.

Step 5: Protect the application in server.js

Make sure these two imports are at the top of `server.js`. Change the paths if your folders are different.

server.js (top of file)

```
import loginController from "./controllers/login.js";
import isLoggedIn from "./tasks/session.js";
```

Then add these two additional routes after the first route, where the static controller is called to serve the CSS file.

server.js

```
if(url.pathname==="/login"){
  return loginController({request})
}

if(!isLoggedIn(request)){
  return new Response(null, {status:302, headers:{Location: "/login"}})
}
```

- The first route shows the login page. It must come first, otherwise the user could never reach it.
- The second route sends everyone who is not logged in to `/login`.

Step 6: Test the application

1. Start the server and open the application in the browser. You should be redirected to the login page.
2. Enter a wrong username or password. You should see an error message.
3. Sign in with the correct details. You should be redirected to the home page.
4. Open the browser developer tools and find the `session_id` cookie under the cookies of your site.

Task

Your application is now protected with a username and password. Find the username and password in the login controller and use them to sign in and see the students' information.